mra.py 中图学习插补模块的科研式说明手册

2026-04-13

摘要

本文面向仓库当前版本的 `mra.py`，系统说明其中与**图学习插补**直接相关的代码、数学原理、训练流程与测试流程。文档重点分析四条主线。第一，图插补模块的代码入口、训练入口与测试入口分别位于何处，真实执行调用链怎样串联。第二，图学习功能如何从训练窗口构造节点动态上下文、距离先验与自适应邻接矩阵，以及这张图如何驱动 GCN 完成窗口内变量关系建模。第三，训练阶段哪些参数会被更新、由哪些损失更新、这些训练得到的参数和统计量在测试集里怎样继续发挥作用。第四，代码中的张量在各子模块之间如何流动，输入输出形状如何变化。文中所有关键结论均以源代码与一次真实运行探针为依据，并额外标注当前实现中的重要细节，例如训练期额外遮挡、图先验的训练集来源、`detector_loss` 对图分支的反向作用，以及 `node_coords` 在当前主程序路径下实际上不参与前向计算这一事实。频域专家与检测器只在解释图模块耦合关系时作必要说明，不展开全部细节。

关键词： 图学习插补，GCN，自适应邻接矩阵，多采样率，门控融合，异常检测

目录

1 文档范围与核心结论 **3**
1.1 分析范围 . . . 3
1.2 先给结论 . . . 3

2 代码入口、训练入口与总体调用链 **4**
2.1 入口总表 . . . 4
2.2 真实调用链 . . . 5
2.3 图专家与其它模块的边界 . . . 5

3 数据、窗口、掩码与训练测试切分 **5**
3.1 默认数据的真实尺寸 . . . 5
3.2 只用观测值拟合的标准化器 . . . 5
3.3 窗口构造方式 . . . 6
3.4 训练集自动推断的多采样率元数据 . . . 6

4 图插补模块的总体结构与张量流 **7**
4.1 训练 batch 中的三种掩码 . . . 7
4.2 整体前向流程 . . . 7
4.3 为什么先做 PreImpute . . . 7
4.4 关键张量形状总表 . . . 8

5 图学习功能的数学原理 8
5.1 图先验首先来自训练集，而不是先验物理拓扑 8
5.2 窗口动态上下文的构造 9
5.3 节点表示、边打分与自适应邻接矩阵 9
5.4 对上述构图公式的文字解释 10
5.5 图正则项的作用 11

6 GCN 重构与双专家门控的代码实现 11
6.1 GCN 重构的数学形式 11
6.2 对 GCN 设计的文字解释 11
6.3 门控层如何把图专家接入最终插补 12

7 训练流程、损失函数与参数更新 12
7.1 训练主循环 12
7.2 五项损失的精确定义 13
7.3 哪些损失更新哪些参数 13
7.4 图模块相关参数清单 13

8 训练集得到的参数和统计量在测试集中的作用 14
8.1 需要区分“梯度学习到的参数”和“训练集估计出的统计量” 14
8.2 训练集到测试集的作用映射 14
8.3 测试阶段图模块怎样影响最终结果 15

9 测试流程、分数构造与图学习结果的输出 15
9.1 测试补全流程 15
9.2 测试评分流程 16
9.3 为什么训练集补全结果还会影响测试打分 16

10 实现层面的设计思路、优点与局限 16
10.1 设计思路总结 16
10.2 当前实现的四个重要细节 16
10.3 从研究复现角度给出的建议 17

11 结论 17

1 文档范围与核心结论

1.1 分析范围

本文只分析与图学习插补主线直接相关的内容：

- `mra.py` 中 `PreImpute`、`TwoExpertGatedImputer`、`FusionAnomalyModel`、`compute_losses`、`train_model`、`reconstruct_full_sequence`、`collect_window_statistics`；
- `graph_gcn_reconstruction.py` 中 `ObservedStandardScaler`、训练集距离先验构造、`GraphLearner`、`GraphGCNReconstructor`、`adjacency_balance_loss`；
- `utils/methods/windowing.py` 与 `utils/methods/data_loading.py` 中决定训练窗口、测试窗口和结构性缺失掩码含义的函数；
- 图专家与频域专家、检测器之间的耦合位置，但不会把频域专家和 Transformer 检测器展开成全文主体。

1.2 先给结论

基于代码阅读和真实运行探针，可以先给出九条最关键的结论：

- 图插补模块的**代码入口**位于 `TwoExpertGatedImputer.forward` 中的

  ```
  x_gcn, adjacency = self.gcn(x_seed, model_missing_mask)
  ```

  这里的 `self.gcn` 实际上是 `GraphGCNReconstructor`。
- 运行 `python mra.py` 时，真正发生参数更新的**训练入口**不是独立实验脚本，而是 `mra.py` 中的 `train_model`；图专家、频域专家、门控网络和检测器共享一个 AdamW 优化器，属于**联合训练**。
- 图学习器不是在全局上学习一张固定邻接矩阵，而是**对每个窗口动态生成一张变量图**。这张图依赖于该窗口内的均值、标准差、最近一次观测值和缺失比例，因此是窗口级、自适应、按样本变化的。
- 在当前 `mra.py` 主程序路径中，图先验矩阵总会先由训练集构造出来，然后传给 `GraphLearner`。因此 `GraphLearner.node_coords` 这组参数虽然被定义为可训练参数，但在当前主路径下并不参与前向，也不会获得梯度。
- 图学习分支的数学核心不是传统固定图卷积，而是“**训练集距离先验 + 窗口动态统计 + 节点相似度**”共同决定的邻接打分函数，再通过温度化 softmax、对称化和行归一化得到邻接矩阵。
- GCN 重构时使用的是**同一窗口共享**、**对时间维共享**的邻接矩阵：时间维没有在图卷积里被卷，图分支真正建模的是“同一时刻不同变量之间的关系”；时间信息主要在构图时通过窗口统计量进入。
- 五项损失对参数的作用范围不同：`gcn_loss` 只更新图分支，`graph_loss` 只更新图学习器，`fusion_loss` 会同时更新图分支、频域分支、门控和采样率嵌入，`detector_loss` 还会进一步反向更新图分支和门控。

- 训练集不仅更新神经网络权重，还生成一批**测试阶段必须继续使用**的非梯度量：标准化器均值方差、训练集驱动的距离先验矩阵、`rate_id`、`stride`、训练分数均值与标准差、异常阈值。
- 当前默认数据下，训练集和测试集均为 4100×18，插补窗口均为 $4100 \times 50 \times 18$，训练集中变量被自动识别为三类采样率：步长 1、3、5。

2 代码入口、训练入口与总体调用链

2.1 入口总表

位置	代码定位	作用
主程序入口	`mra.py :862--1122`	读取训练/测试数据、标准化、构建窗口、推断采样率元数据、训练模型、补全序列、统计分数、阈值化并保存结果。
采样率元数据入口	`mra.py :150--161`	根据训练集缺失率推断每个变量的 `stride` 和 `rate_id`。
训练期额外遮挡	`mra.py :173--193`	在可观测位置上随机采样 `holdout_mask`，构造自监督插补目标。
预插补入口	`mra.py :196--259`	通过双向填充生成 $\boldsymbol{X}_{\text{seed}}$，为图专家与频域专家提供平滑输入。
图专家代码入口	`mra.py :317--319`	在 `TwoExpertGatedImputer.forward` 中调用 `GraphGCNReconstructor`。
门控融合入口	`mra.py :325--343`	拼接图专家、频域专家与采样率嵌入，输出门控权重和最终插补结果。
联合模型封装	`mra.py :470--524`	用 `FusionAnomalyModel` 串起插补器与检测器。
损失定义入口	`mra.py :527--560`	定义 `fusion/gcn/freq/detector/graph` 五项损失和总损失。
真实训练入口	`mra.py :563--636`	前向、损失、反向传播、梯度裁剪、优化器更新都在这里完成。
测试补全入口	`mra.py :639--677`	用训练后的模型输出补全序列、门控权重和邻接矩阵样本。
测试评分入口	`mra.py :680--737`	计算 `detector_score`、`disagreement_score` 和 `gate_entropy`。
训练集距离先验	`graph_gcn_ reconstruction.py :228--273`	用训练数据轨迹和观测模式构造变量之间的欧氏距离矩阵。
图学习定义	`graph_gcn_ reconstruction.py :350--458`	定义 `GraphLearner`，根据窗口动态上下文生成邻接矩阵。
GCN 重构定义	`graph_gcn_ reconstruction.py :463--506`	定义 `GraphGCNReconstructor`，执行三层图消息传递重构。

2.2 真实调用链

当用户运行 `python mra.py` 时，与图插补最相关的真实执行链条为

$$
\begin{aligned}
\texttt{main} &\to \texttt{train_model} \to \texttt{FusionAnomalyModel.forward} \\
&\to \texttt{TwoExpertGatedImputer.forward} \\
&\to \texttt{PreImpute.fill} \\
&\to \texttt{GraphGCNReconstructor.forward} \\
&\to \texttt{GraphLearner.forward} \\
&\to \texttt{compute_losses} \to \texttt{loss.backward} \to \texttt{AdamW.step}.
\end{aligned}
\tag{1}
$$

这条链路有两个含义：

1. 图专家不是独立于整套模型外部训练的，而是被包进联合模型内部；
2. 图分支的训练结果不只影响插补误差，还会通过门控和检测器继续影响最终异常分数。

2.3 图专家与其它模块的边界

虽然图学习是本文重点，但当前实现中的图专家并不单独闭环。它的输出 $\boldsymbol{X}_{\text{gcn}}$ 至少会走向三条下游路径：

- 与频域专家竞争门控权重，参与 $\boldsymbol{X}_{\text{imputed}}$ 和 $\boldsymbol{X}_{\text{complete}}$；
- 与频域专家做差，构成 `disagreement_score`；
- 通过 $\boldsymbol{X}_{\text{complete}}$ 进入检测器，间接影响 `detector_score` 和后续阈值化结果。

因此要真正理解图学习模块，必须把它放在“图专家 + 门控 + 检测器”的联合训练系统中分析，而不是只盯住 `GraphLearner` 自身。

3 数据、窗口、掩码与训练测试切分

3.1 默认数据的真实尺寸

对仓库当前默认数据，代码读取后得到：

$$\boldsymbol{X}_{\text{raw}}^{\text{train}} \in \mathbb{R}^{4100\times 18}, \tag{2}$$

$$\boldsymbol{X}_{\text{raw}}^{\text{test}} \in \mathbb{R}^{4100\times 18}, \tag{3}$$

$$\boldsymbol{M}_s^{\text{train}}, \boldsymbol{M}_s^{\text{test}} \in \{0,1\}^{4100\times 18}. \tag{4}$$

其中 $\boldsymbol{M}_s$ 是**结构性缺失掩码**，也就是 CSV 中原生的 `NaN` 位置。当前训练集和测试集的总体缺失率都约为 0.4889。

3.2 只用观测值拟合的标准化器

`ObservedStandardScaler` 只用训练集观测值拟合每个变量的均值 μ_n 与标准差 σ_n，然后对训练集和测试集都做同样的变换：

$$X_{t,n} = \begin{cases} \dfrac{X_{t,n}^{\text{raw}} - \mu_n}{\sigma_n}, & M_{s,t,n} = 0, \\ 0, & M_{s,t,n} = 1. \end{cases} \tag{5}$$

这意味着标准化后的数值零有两层可能语义：

- 某个观测值在标准化后恰好等于零；
- 某个位置原本缺失，被训练集均值填充后标准化成零。

因此后续模型必须同时读取数值张量和掩码张量，不能只看数值本身。

3.3 窗口构造方式

当前代码中有三类窗口：

- `build_windows`：用于训练插补与完整序列补全，允许在序列前端重复首行进行补齐；
- `build_standard_windows`：用于训练阶段统计异常分数，不做前端补齐；
- `build_prompt_test_windows`：用于测试阶段统计异常分数，同时生成固定标签。

默认 `seq_len=50`、`stride=1` 时，真实形状为

$$\boldsymbol{W}_{\text{imp}}^{\text{train}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{6}$$

$$\boldsymbol{W}_{\text{imp}}^{\text{test}} \in \mathbb{R}^{4100\times 50\times 18}, \tag{7}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{train}} \in \mathbb{R}^{4001\times 50\times 18}, \tag{8}$$

$$\boldsymbol{W}_{\text{eval}}^{\text{test}} \in \mathbb{R}^{4000\times 50\times 18}. \tag{9}$$

其中训练插补窗口与测试插补窗口都覆盖全序列，因此 `reconstruct_full_sequence` 只要取每个窗口的最后一个时间步，就能对齐还原出长度为 4100 的完整序列。

3.4 训练集自动推断的多采样率元数据

`infer_rate_metadata` 根据每列缺失率估计观测率

$$r_n = 1 - \frac{1}{T_{\text{all}}} \sum_{t=1}^{T_{\text{all}}} M_{s,t,n}, \tag{10}$$

并据此得到采样步长

$$s_n = \text{round}\left(\frac{1}{\max(r_n, 10^{-6})}\right). \tag{11}$$

对当前默认数据，18 个变量被分为三组：

变量组	列索引	缺失率	观测率	推断步长 s_n	`rate_id`
高频组	1–6	0.0	1.0	1	0
中频组	7–12	0.6667	0.3333	3	1
低频组	13–18	0.8	0.2	5	2

因此主程序中会固定使用

$$\boldsymbol{stride} = [1,1,1,1,1,1,3,3,3,3,3,3,5,5,5,5,5,5], \tag{12}$$

$$\boldsymbol{rate_id} = [0,0,0,0,0,0,1,1,1,1,1,1,2,2,2,2,2,2]. \tag{13}$$

需要特别指出：图学习器本身**并不显式读取 $\boldsymbol{rate_id}$** 或 $\boldsymbol{stride}$。图分支只读取 $\boldsymbol{X}_{\text{seed}}$ 与掩码，因此它对多采样率的感知是**隐式的**，主要来自缺失模式和窗口统计；显式采样率嵌入只出现在门控层与检测器中。

4 图插补模块的总体结构与张量流

4.1 训练 batch 中的三种掩码

对一个训练 batch，记真实标准化窗口为

$$\boldsymbol{X}_{\text{true}} \in \mathbb{R}^{B\times T\times N}, \tag{14}$$

其中默认 $T = 50$、$N = 18$。代码中会同时出现三种掩码：

- $\boldsymbol{M}_s$：结构性缺失掩码，来自原始数据；
- $\boldsymbol{M}_h$：训练期额外遮挡掩码，只从可观测位置里抽样；
- $\boldsymbol{M}$：模型输入掩码，满足 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$。

于是模型真正接收到的输入窗口为

$$\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M}). \tag{15}$$

`sample_holdout_mask` 还包含一个非常重要的保底逻辑：如果某个样本窗口里本来有观测值，但按伯努利采样没有抽到任何 holdout 点，那么代码会**强行随机挑一个观测点做 holdout**。这保证了插补损失在该样本上不会完全失去监督。

4.2 整体前向流程

图插补与其上下游的关系可以写成

$$\begin{aligned}
\boldsymbol{X}_{\text{seed}} &= \text{PreImpute}(\boldsymbol{X}_{\text{input}}, \boldsymbol{M}),\\
\boldsymbol{X}_{\text{gcn}}, \boldsymbol{A} &= f_{\text{graph}}(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}),\\
\boldsymbol{X}_{\text{freq}} &= f_{\text{freq}}(\boldsymbol{X}_{\text{seed}}),\\
\boldsymbol{\Pi} &= g(\boldsymbol{X}_{\text{seed}}, \boldsymbol{M}, \boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{freq}}, \boldsymbol{rate_id}),\\
\boldsymbol{X}_{\text{imputed}} &= \Pi_{:,:,:,0} \odot \boldsymbol{X}_{\text{gcn}} + \Pi_{:,:,:,1} \odot \boldsymbol{X}_{\text{freq}},\\
\boldsymbol{X}_{\text{complete}} &= \boldsymbol{M} \odot \boldsymbol{X}_{\text{imputed}} + (1 - \boldsymbol{M}) \odot \boldsymbol{X}_{\text{input}}.
\end{aligned} \tag{16}$$

从这个公式可以直接看出三点：

- 图专家输出的是 $\boldsymbol{X}_{\text{gcn}}$ 与邻接矩阵 $\boldsymbol{A}$；
- 图专家不是直接生成最终结果，而是与频域专家一起被门控层加权；
- 在非缺失位置，$\boldsymbol{X}_{\text{complete}}$ 保持原始输入值；只有在结构性缺失或 holdout 位置，模型才会写入插补结果。

4.3 为什么先做 PreImpute

`PreImpute.fill` 使用双向时域填充生成 $\boldsymbol{X}_{\text{seed}}$。这一步的目的不是做最终插补，而是给图专家和频域专家提供一个更平滑、可处理的种子输入。它的规则是：

- 若当前位置可观测，直接保留原值；

- 若缺失且前后两侧都有观测值，则取正向最近观测值与反向最近观测值的均值；
- 若仅一侧存在可用观测值，则取单侧填充值；
- 若整条可见历史都不存在，则保持零。

这一策略是**双向最近观测均值填充**，不是严格按时间位置做线性插值。举例而言，对单变量片段 [1, ?, ?, 4]，该模块会得到 [1, 2.5, 2.5, 4]，而不是 [1, 2, 3, 4]。

4.4 关键张量形状总表

张量	默认形状	说明
$\boldsymbol{X}_{\text{true}}$	$\mathbb{R}^{B\times 50\times 18}$	训练或测试窗口，值已按训练集统计量标准化。
$\boldsymbol{M}_s, \boldsymbol{M}_h, \boldsymbol{M}$	$\mathbb{R}^{B\times 50\times 18}$	结构性缺失、额外遮挡、模型输入掩码。
$\boldsymbol{X}_{\text{input}}$	$\mathbb{R}^{B\times 50\times 18}$	经过 $\boldsymbol{M}$ 置零后的模型输入。
$\boldsymbol{X}_{\text{seed}}$	$\mathbb{R}^{B\times 50\times 18}$	预插补结果，图专家与频域专家的共同输入。
节点动态上下文	$\mathbb{R}^{B\times 18\times 4}$	依次为均值、标准差、最近一次观测值、缺失比例。
节点静态上下文	$\mathbb{R}^{B\times 18\times 8}$	可训练的节点静态向量 `static_context` 复制到 batch 维。
节点表示	$\mathbb{R}^{B\times 18\times 32}$	`node_encoder` 输出。
边特征	$\mathbb{R}^{B\times 18\times 18\times 128}$	由 $[h_i, h_j, \lvert h_i - h_j\rvert, h_i \odot h_j]$ 拼接得到。
邻接矩阵 $\boldsymbol{A}$	$\mathbb{R}^{B\times 18\times 18}$	每个窗口一张变量图。
GCN 第一层输出	$\mathbb{R}^{B\times 50\times 18\times 32}$	`gcn_in` 后的隐藏表示。
GCN 第二层输出	$\mathbb{R}^{B\times 50\times 18\times 32}$	`gcn_mid` 后的隐藏表示。
$\boldsymbol{X}_{\text{gcn}}$	$\mathbb{R}^{B\times 50\times 18}$	图专家重构输出。
$\boldsymbol{X}_{\text{freq}}$	$\mathbb{R}^{B\times 50\times 18}$	频域专家重构输出。
门控输入	$\mathbb{R}^{B\times 50\times 18\times 14}$	6 个标量特征加 8 维采样率嵌入。
门控权重 $\boldsymbol{\Pi}$	$\mathbb{R}^{B\times 50\times 18\times 2}$	最后一维对应图专家和频域专家。
$\boldsymbol{X}_{\text{imputed}}$	$\mathbb{R}^{B\times 50\times 18}$	两个专家的加权和。
$\boldsymbol{X}_{\text{complete}}$	$\mathbb{R}^{B\times 50\times 18}$	用于下游检测器的完整窗口。

5 图学习功能的数学原理

5.1 图先验首先来自训练集，而不是先验物理拓扑

若用户没有提供外部物理坐标或距离矩阵，主程序会调用 `resolve_distance_prior_matrix`，用训练集自动生成一个变量之间的距离矩阵。对第 n 个变量，代码构造描述向量

$$\boldsymbol{u}_n = \left[\boldsymbol{x}^{\text{train}}_{:,n}; \lambda \boldsymbol{o}^{\text{train}}_{:,n}\right] \in \mathbb{R}^{2T_{\text{all}}}, \tag{17}$$

其中 $\boldsymbol{x}^{\text{train}}_{:,n}$ 是训练集标准化轨迹，$\boldsymbol{o}^{\text{train}}_{:,n} = 1 - \boldsymbol{M}^{\text{train}}_{s,:,n}$ 是该变量的观测模式，默认 $\lambda = 0.5$。

然后定义变量间距离

$$D_{n,m} = \left\|\boldsymbol{u}_n - \boldsymbol{u}_m\right\|_2 . \tag{18}$$

这一步有两个非常重要的含义：

- 默认图先验并不是管道布置、空间坐标或工艺流程图，而是**训练集驱动的变量相似性先验**；
- 该先验同时看轨迹相似度和缺失模式相似度，因此天然把多采样率信息的一部分折进了图先验。

`GraphLearner` 进一步把距离矩阵变成相似度先验

$$P_{n,m} = \frac{1}{1 + D_{n,m}/\eta}, \tag{19}$$

其中 η 是距离矩阵正元素的中位数。由于对角线距离被强制置零，这个先验只作用于非对角项。

5.2 窗口动态上下文的构造

图学习器并不直接拿整个 50×18 的窗口去生成边，而是先把每个变量在当前窗口中的信息压缩成 4 维动态统计量。对 batch 中第 b 个样本、变量 n，定义观测掩码

$$O_{b,t,n} = 1 - M_{b,t,n}. \tag{20}$$

再定义观测数

$$c_{b,n} = \sum_{t=1}^{T} O_{b,t,n}, \tag{21}$$

并在代码中使用 $\max(c_{b,n}, 1)$ 防止除零。随后可得：

$$\mu_{b,n} = \frac{\sum_{t=1}^{T} X_{\text{seed},b,t,n} O_{b,t,n}}{\max(c_{b,n}, 1)}, \tag{22}$$

$$\sigma_{b,n} = \sqrt{\frac{\sum_{t=1}^{T} \left(X_{\text{seed},b,t,n} - \mu_{b,n}\right)^2 O_{b,t,n}}{\max(c_{b,n}, 1)} + 10^{-6}}, \tag{23}$$

$$\ell_{b,n} = \text{当前窗口中最近一次观测值}, \tag{24}$$

$$\rho_{b,n} = \frac{1}{T} \sum_{t=1}^{T} M_{b,t,n}. \tag{25}$$

于是节点动态上下文为

$$\boldsymbol{d}_{b,n} = [\mu_{b,n}, \sigma_{b,n}, \ell_{b,n}, \rho_{b,n}] \in \mathbb{R}^4. \tag{26}$$

这个设计体现了当前图学习器的思想：**用窗口级统计量去概括变量在该窗口中的状态，再据此判断变量间连接强度**。也就是说，图不是由单个时间点的瞬时观测决定，而是由“最近一段时间内该变量的统计行为”决定。

5.3 节点表示、边打分与自适应邻接矩阵

除动态上下文外，`GraphLearner` 还为每个变量保存一个可训练静态向量 $\boldsymbol{s}_n \in \mathbb{R}^8$。于是节点输入为

$$\boldsymbol{v}_{b,n} = [\boldsymbol{d}_{b,n}; \boldsymbol{s}_n] \in \mathbb{R}^{12}. \tag{27}$$

通过两层 MLP 得到节点表示

$$\boldsymbol{h}_{b,n} = \phi_\theta(\boldsymbol{v}_{b,n}) \in \mathbb{R}^{32}. \tag{28}$$

然后对任意节点对 (n, m)，构造边特征

$$\boldsymbol{p}_{b,n,m} = [\boldsymbol{h}_{b,n}; \boldsymbol{h}_{b,m}; |\boldsymbol{h}_{b,n} - \boldsymbol{h}_{b,m}|; \boldsymbol{h}_{b,n} \odot \boldsymbol{h}_{b,m}] \in \mathbb{R}^{128}, \tag{29}$$

并用 `edge_mlp` 产生边打分

$$E_{b,n,m} = \psi_\theta(\boldsymbol{p}_{b,n,m}). \tag{30}$$

与此同时，代码还额外加入了节点表示的余弦相似度

$$C_{b,n,m} = \frac{\boldsymbol{h}_{b,n}^\top \boldsymbol{h}_{b,m}}{\|\boldsymbol{h}_{b,n}\|_2 \|\boldsymbol{h}_{b,m}\|_2}, \tag{31}$$

并通过三个可训练标量控制先验、相似度和温度的强弱：

$$\begin{aligned} \alpha &= \mathrm{softplus}(\texttt{prior_strength}), \\ \beta &= \mathrm{softplus}(\texttt{similarity_strength}), \\ \tau &= \mathrm{softplus}(\texttt{temperature}) + 10^{-4}. \end{aligned} \tag{32}$$

令 $\gamma = \mathrm{softplus}(\texttt{self_loop_logit})$，则未归一化邻接 logit 可写成

$$Z_{b,n,m} = \begin{cases} E_{b,n,m} + \alpha P_{n,m} + \beta C_{b,n,m}, & n \neq m, \\ \gamma, & n = m. \end{cases} \tag{33}$$

代码随后进行三步变换：

$$\tilde{A}_{b,n,m} = \mathrm{softmax}_m\left(\frac{Z_{b,n,m}}{\tau}\right), \tag{34}$$

$$\bar{A}_b = \frac{1}{2}\left(\tilde{A}_b + \tilde{A}_b^\top\right), \tag{35}$$

$$A_{b,n,m} = \frac{\bar{A}_{b,n,m}}{\sum_j \bar{A}_{b,n,j}}. \tag{36}$$

5.4 对上述构图公式的文字解释

上面的公式虽然短，但设计含义很多，必须逐条解释：

1. **边打分不是只靠一个来源**。当前实现把边打分拆成三部分：数据驱动的边 MLP 输出 $E_{b,n,m}$、训练集距离先验 $P_{n,m}$ 与窗口内节点表示余弦相似度 $C_{b,n,m}$。这让图既保留训练集结构偏好，又能根据当前窗口动态调整。

2. **先验项是窗口不变的，动态项是窗口变化的**。$P_{n,m}$ 对所有窗口共享，而 $E_{b,n,m}$ 与 $C_{b,n,m}$ 随当前窗口统计量变化。因此最终图是“固定先验 + 动态校正”的混合体。

3. **softplus 使权重始终为正**。这意味着代码不允许 α、β、τ 和 γ 取负数。直观地说，先验和相似度只能增强贡献强度，温度始终为正，自环 logit 始终为正。

4. **softmax 让每个节点把权重分配给所有节点**。行 softmax 后，每一行都变成非负且和为 1 的分布，因此每个节点都像是在决定“我应该从哪些变量吸收信息、各吸多少”。

5. **对称化不是最终精确约束，而是中间修正**。代码先把矩阵与其转置取平均，再做逐行归一化，因此最终输出矩阵是**近似对称且严格行归一化**的；最后一步后，严格意义上的完全对称一般不再精确成立。

6. **图在时间维上是共享的**。当前窗口里所有 50 个时间步共用同一张邻接矩阵。换句话说，图学习器认为“变量关系在这个窗口内近似稳定”，而不是每个时间步重新学一张图。

5.5 图正则项的作用

`adjacency_balance_loss` 只约束邻接矩阵对角元素接近目标值 δ：

$$\mathcal{L}_{\text{graph}} = \frac{1}{BN}\sum_{b=1}^{B}\sum_{n=1}^{N}\left(A_{b,n,n}-\delta\right)^2, \tag{37}$$

默认 $\delta = 0.25$。

由于每一行都归一化为 1，这个目标值的直观意义是：平均来说，希望每个节点保留约 25% 的自环权重，把剩余 75% 分配给其他变量。该正则并不直接鼓励稀疏性，也不直接约束拉普拉斯平滑性；它的主要作用是防止图退化成过强自环或过弱自环。

6 GCN 重构与双专家门控的代码实现

6.1 GCN 重构的数学形式

`GraphGCNReconstructor` 先用 `GraphLearner` 得到 $\boldsymbol{A}$，然后进行三层图消息传递。令

$$\boldsymbol{H}^{(0)} = \boldsymbol{X}_{\text{seed}} \in \mathbb{R}^{B\times T\times N}, \tag{38}$$

并把其最后一维扩展成长度为 1 的特征维：

$$\boldsymbol{H}^{(0)\prime} \in \mathbb{R}^{B\times T\times N\times 1}. \tag{39}$$

`GCNLayer` 的计算方式是“先线性映射，再沿节点维用邻接矩阵聚合”。对任一层 l，其核心形式为

$$\tilde{\boldsymbol{H}}^{(l)}_{b,t,n,:} = \sum_{m=1}^{N} A_{b,n,m}\left(\boldsymbol{H}^{(l-1)}_{b,t,m,:}\boldsymbol{W}^{(l)} + \boldsymbol{b}^{(l)}\right). \tag{40}$$

第一层和第二层还会经过 ReLU + LayerNorm + Dropout：

$$\boldsymbol{H}^{(1)} = \text{Dropout}\Big(\text{LayerNorm}\Big(\text{ReLU}(\tilde{\boldsymbol{H}}^{(1)})\Big)\Big), \tag{41}$$

$$\boldsymbol{H}^{(2)} = \text{Dropout}\Big(\text{LayerNorm}\Big(\text{ReLU}(\tilde{\boldsymbol{H}}^{(2)})\Big)\Big). \tag{42}$$

最后一层输出

$$\boldsymbol{X}_{\text{gcn}} = \tilde{\boldsymbol{H}}^{(3)} \in \mathbb{R}^{B\times T\times N}. \tag{43}$$

6.2 对 GCN 设计的文字解释

这里的 GCN 与很多图时序模型不同，重点有三条：

- 图卷积只在**变量维**上传播，不在时间维上传播。因此它学习的是“同一时刻不同变量之间如何相互借力完成重构”。
- 时间信息没有完全消失，而是通过图学习器中的窗口统计量间接参与构图。也就是说，时间先用于**决定图**，图再用于**传播变量信息**。
- 由于当前 $N = 18$ 很小，显式构造 $N \times N$ 的边特征和邻接矩阵计算开销是可接受的；如果变量数显著增加，该设计的二次复杂度会迅速抬升。

6.3 门控层如何把图专家接入最终插补

图专家并不是直接输出最终补全结果，而是与频域专家一起交给门控层决定权重。门控输入特征为

$$\boldsymbol{g}_{b,t,n} = \left[X_{\text{seed},b,t,n}, M_{b,t,n}, X_{\text{gcn},b,t,n}, X_{\text{freq},b,t,n}, X_{\text{gcn},b,t,n} - X_{\text{seed},b,t,n}, X_{\text{freq},b,t,n} - X_{\text{seed},b,t,n}, \boldsymbol{e}_n^{\text{rate}}\right], \tag{44}$$

其中 $\boldsymbol{e}_n^{\text{rate}} \in \mathbb{R}^8$ 为采样率嵌入，因此默认总维度为 $6 + 8 = 14$。

经过两层 GELU MLP 后得到两位专家的 logit，再用 softmax 生成门控权重

$$\boldsymbol{\pi}_{b,t,n} = \text{softmax}(\text{MLP}_{\text{gate}}(\boldsymbol{g}_{b,t,n})) \in \mathbb{R}^2. \tag{45}$$

于是

$$X_{\text{imputed},b,t,n} = \pi_{b,t,n,0} X_{\text{gcn},b,t,n} + \pi_{b,t,n,1} X_{\text{freq},b,t,n}. \tag{46}$$

最终完整窗口满足

$$X_{\text{complete},b,t,n} = \begin{cases} X_{\text{input},b,t,n}, & M_{b,t,n} = 0, \\ X_{\text{imputed},b,t,n}, & M_{b,t,n} = 1. \end{cases} \tag{47}$$

这说明图专家对最终输出的影响并非“非黑即白”，而是由门控层按位置逐点决定的。

7 训练流程、损失函数与参数更新

7.1 训练主循环

`train_model` 的每个 batch 都执行如下步骤：

1. 从 $\boldsymbol{W}_{\text{imp}}^{\text{train}}$ 取出 $\boldsymbol{X}_{\text{true}}$ 和结构性缺失掩码 $\boldsymbol{M}_s$；
2. 在观测位置上抽样 $\boldsymbol{M}_h$，形成 $\boldsymbol{M} = \max(\boldsymbol{M}_s, \boldsymbol{M}_h)$；
3. 构造 $\boldsymbol{X}_{\text{input}} = \boldsymbol{X}_{\text{true}} \odot (1 - \boldsymbol{M})$；
4. 前向得到 $\boldsymbol{X}_{\text{gcn}}$、$\boldsymbol{A}$、$\boldsymbol{X}_{\text{freq}}$、$\boldsymbol{X}_{\text{complete}}$ 和检测器重构结果；
5. 计算五项损失并求和；
6. `zero_grad`，然后对总损失做 `backward`；
7. 对全模型做 ℓ_2 范数为 1 的梯度裁剪；
8. 用 AdamW 更新参数。

默认优化器为

$$\text{AdamW}(\texttt{lr} = 10^{-3},\ \texttt{weight_decay} = 10^{-4}). \tag{48}$$

7.2 五项损失的精确定义

设 $\boldsymbol{H} = \boldsymbol{M}_h$ 为 holdout 掩码，则三项插补监督损失均只在 holdout 位置上计算：

$$\mathcal{L}_{\text{fusion}} = \text{MSE}(\boldsymbol{X}_{\text{imputed}}, \boldsymbol{X}_{\text{true}}; \boldsymbol{H}), \tag{49}$$

$$\mathcal{L}_{\text{gcn}} = \text{MSE}(\boldsymbol{X}_{\text{gcn}}, \boldsymbol{X}_{\text{true}}; \boldsymbol{H}), \tag{50}$$

$$\mathcal{L}_{\text{freq}} = \text{MSE}(\boldsymbol{X}_{\text{freq}}, \boldsymbol{X}_{\text{true}}; \boldsymbol{H}). \tag{51}$$

图正则项为前节给出的

$$\mathcal{L}_{\text{graph}} = \frac{1}{BN} \sum_{b,n} \left(A_{b,n,n} - \delta\right)^2. \tag{52}$$

检测器损失定义为

$$\mathcal{L}_{\text{det}} = \text{MSE}\left(\boldsymbol{X}_{\text{det_rec}}, \text{stopgrad}(\boldsymbol{X}_{\text{complete}}); 1 - \boldsymbol{M}_s\right). \tag{53}$$

这里有两个实现细节必须明确写出来：

- 掩码用的是 $1 - \boldsymbol{M}_s$，不是 $1 - \boldsymbol{M}$。因此**训练期 holdout 位置仍然会参与检测器损失**；
- 目标张量 $\boldsymbol{X}_{\text{complete}}$ 做了 `detach()`，但输入分支没有被截断，因此 $\mathcal{L}_{\text{det}}$ 仍会反向更新插补器。

总代价函数为

$$\mathcal{L} = 1.0\,\mathcal{L}_{\text{fusion}} + 0.4\,\mathcal{L}_{\text{gcn}} + 0.4\,\mathcal{L}_{\text{freq}} + 0.5\,\mathcal{L}_{\text{det}} + 0.1\,\mathcal{L}_{\text{graph}}. \tag{54}$$

7.3 哪些损失更新哪些参数

对当前代码做单 batch 梯度探针后，可以得到清晰的更新范围：

损失项	图学习器	GCN 层	频域专家	门控 MLP	采样率嵌入	检测器
$\mathcal{L}_{\text{fusion}}$	是	是	是	是	是	否
$\mathcal{L}_{\text{gcn}}$	是	是	否	否	否	否
$\mathcal{L}_{\text{freq}}$	否	否	是	否	否	否
$\mathcal{L}_{\text{det}}$	是	是	是	是	是	是
$\mathcal{L}_{\text{graph}}$	是	否	否	否	否	否

这张表意味着：

- 图专家至少受到三种力共同作用：$\mathcal{L}_{\text{gcn}}$ 的直接重构监督、$\mathcal{L}_{\text{fusion}}$ 的门控融合监督，以及 $\mathcal{L}_{\text{det}}$ 通过完整窗口传来的下游可重构性压力；
- $\mathcal{L}_{\text{graph}}$ 不会直接训练 GCN 权重，它只训练 `GraphLearner` 自身输出更符合对角占比目标的图；
- 门控层不会被 $\mathcal{L}_{\text{gcn}}$ 或 $\mathcal{L}_{\text{freq}}$ 单独训练，它主要由 $\mathcal{L}_{\text{fusion}}$ 和 $\mathcal{L}_{\text{det}}$ 决定。

7.4 图模块相关参数清单

当前默认配置下，全模型共有 693,260 个参数，其中与图插补直接相关的主要参数如下：

参数组	参数量	当前主路径是否实际更新	作用
GraphLearner	5925	是，但 node_coords 例外	生成窗口级邻接矩阵。
GCN 三层与两层归一化	1281	是	按邻接矩阵在变量维上传播信息并输出 $\boldsymbol{X}_{\text{gcn}}$。
门控 MLP	5250	是	决定图专家与频域专家在每个位置的权重。
插补器采样率嵌入	24	是	给门控层提供显式采样率信息。
频域专家	24314	是	与图专家共同完成插补。
检测器	656466	是	对 $\boldsymbol{X}_{\text{complete}}$ 做重构并产生异常分数。

GraphLearner 内部又可以细分为：

- 标量门控参数：self_loop_logit、prior_strength、similarity_strength、temperature；
- 节点静态上下文：static_context；
- 节点表示网络：node_encoder；
- 边打分网络：edge_mlp；
- node_coords。

其中 node_coords 有 144 个参数，但在当前主程序路径中不会更新。原因很直接：main 总会先调用 resolve_distance_prior_matrix 得到一个非空距离矩阵，并把它传给 GraphLearner；此时 _build_prior_adjacency 只使用该固定矩阵，不再读取 node_coords。

8 训练集得到的参数和统计量在测试集中的作用

8.1 需要区分“梯度学习到的参数”和“训练集估计出的统计量”

测试阶段真正起作用的量可以分成两类：

1. **通过反向传播学习到的参数**：图学习器、GCN、门控层、频域专家、检测器的权重；
2. **仅由训练集估计得到的固定量**：标准化均值方差、距离先验矩阵、rate_id、stride、训练分数均值方差、异常阈值。

把这两类混为一谈会直接导致对测试流程的理解错误。

8.2 训练集到测试集的作用映射

训练期得到的量	来源	在测试集中的作用
μ_n, σ_n	ObservedStandardScaler.fit(train_raw, train_mask)	用同一套训练集统计量标准化测试集，并在输出 test_completed.csv 前做逆变换。
训练集驱动距离矩阵 $\boldsymbol{D}$	resolve_distance_prior_matrix(train_scaled, train_mask)	作为图先验输入 GraphLearner，影响测试窗口中每一张邻接矩阵的生成。

训练期得到的量	来源	在测试集中的作用
$\boldsymbol{rate_id}, \boldsymbol{stride}$	`infer_rate_metadata(train_mask)`	在测试阶段继续作为门控层采样率嵌入和检测器相位编码的依据。
图学习器、GCN、门控、频域专家、检测器参数	`train_model` 中 AdamW 训练得到	决定测试阶段的 $\boldsymbol{X}_{\text{gcn}}$、门控权重、$\boldsymbol{X}_{\text{complete}}$、检测器重构和异常分数。
`train_completed_scaled` 及其窗口特征统计	训练集补全结果再窗口化后计算	用于构造 `shift_score` 的训练参考分布，并把同一套统计量应用到测试补全窗口。
训练分数均值方差	`normalize_scores(train, test)`	把测试分数变成以训练集为基准的标准化分数。
异常阈值	`choose_threshold(train_scores, ...)`	对测试最终分数做二值化，得到异常判定。

8.3 测试阶段图模块怎样影响最终结果

测试阶段虽然不再更新参数，但图模块仍通过三条路径持续发挥作用：

- **路径 1：** 通过 $\boldsymbol{X}_{\text{gcn}}$ 参与门控融合，影响 $\boldsymbol{X}_{\text{complete}}$，进而影响检测器重构误差和 `shift_score`；
- **路径 2：** 直接与频域专家做差，形成 `disagreement_score`；
- **路径 3：** 通过测试窗口动态生成邻接矩阵，改变图消息传递方式，从而影响补全结果和门控权重分布。

从最终打分公式看，

$$s_{\text{final}} = |z_{\text{det}}| + \lambda_1 z_{\text{gap}} + \lambda_2 z_{\text{shift}}, \tag{55}$$

其中 z_{det}、z_{gap} 和 z_{shift} 分别表示以训练集均值和标准差归一化后的 `detector_score`、`disagreement_score` 和 `shift_score`。图模块至少影响前两项，并通过补全结果进一步影响第三项。

9 测试流程、分数构造与图学习结果的输出

9.1 测试补全流程

`reconstruct_full_sequence` 在测试阶段做的事情是：

1. 不再使用随机 holdout，只把结构性缺失位置置零；
2. 前向运行整套模型；
3. 取每个窗口最后一个时间步的 $\boldsymbol{X}_{\text{complete}}$，拼成完整序列；
4. 同时保存每个窗口最后一个时间步的门控权重，以及整张邻接矩阵样本。

因此输出文件中：

- `test_completed.csv` 反映的是图专家与频域专家在真实测试缺失模式下的联合补全结果；
- `train_gate_weights.csv` 与 `test_gate_weights.csv` 反映图专家在不同变量上的平均权重；
- `train_adjacency_heatmap.png` 反映训练窗口邻接矩阵的平均图结构。

9.2 测试评分流程

`collect_window_statistics` 会为训练评分窗口和测试评分窗口都计算三类统计量：

$$\texttt{detector_score} = \frac{\sum(\boldsymbol{X}_{\text{det_rec}} - \boldsymbol{X}_{\text{complete}})^2 \odot (1 - \boldsymbol{M}_s)}{\sum(1 - \boldsymbol{M}_s)}, \tag{56}$$

$$\texttt{disagreement_score} = \operatorname{mean} |\boldsymbol{X}_{\text{gcn}} - \boldsymbol{X}_{\text{freq}}|, \tag{57}$$

$$\texttt{gate_entropy} = -\operatorname{mean} \sum_{k=1}^{2} \Pi_k \log \Pi_k. \tag{58}$$

其中 `disagreement_score` 是最直接体现图专家作用的测试统计量之一：如果图专家与频域专家在某些窗口上出现持续高分歧，那么这本身会被纳入最终异常分数。

9.3 为什么训练集补全结果还会影响测试打分

主程序在模型训练完毕后，还会对训练集和测试集分别做一次完整补全，再将这些补全后的窗口送入 `compute_distribution_shift_scores`。也就是说，`shift_score` 不是从原始带缺失窗口上计算出来的，而是从**补全后的窗口分布**上计算出来的。

于是图模块对测试 `shift_score` 的影响来自两个方向：

- 它决定训练补全窗口的参考分布是什么；
- 它决定测试补全窗口相对该参考分布偏移多大。

这也解释了为什么图插补模块虽然名义上是“插补前端”，却会显著影响最终异常检测性能。

10 实现层面的设计思路、优点与局限

10.1 设计思路总结

从代码实现看，图学习部分的设计思路可以概括成一句话：

> 用训练集驱动的变量关系先验稳定图结构，用当前窗口的统计行为动态修正图结构，再用该图在变量维上传播信息完成插补。

这套设计相比“固定邻接 + 固定 GCN”的优势主要有三点：

- 图能够随窗口动态变化，更适合异常工况下关系漂移的场景；
- 先验矩阵提供了稳定的结构偏置，降低纯数据驱动构图的随机性；
- 图专家与频域专家通过门控互补，使图学习不必独自承担所有插补难点。

10.2 当前实现的四个重要细节

结合代码，可以额外指出四个容易被忽略但很重要的实现事实：

1. **图先验默认来自训练数据而非物理拓扑**。只有用户显式提供物理坐标或距离矩阵时，先验才会切换成外部物理先验。
2. **图是窗口级的，不是时间步级的**。这会降低复杂度，但也意味着窗口内部若变量关系快速变化，单张图可能不够细。

3. **图正则只约束对角占比**。当前没有显式稀疏约束，也没有图拉普拉斯正则，因此图结构的其它性质主要由数据误差和 softmax 温度间接塑造。

4. `node_coords` **在当前主路径下是失活参数**。这不是概念上的设计缺陷，但在实现上确实意味着有一组定义出的参数实际上没有被训练。

10.3 从研究复现角度给出的建议

如果后续需要把该图模块发展为更完整的科研实现，优先可考虑以下方向：

- 给 `GraphLearner` 增加显式稀疏约束或 top-k 边保留机制，提升图的可解释性；
- 若希望 `node_coords` 发挥作用，可在训练集自动先验存在时把它作为可学习残差项加入先验，而不是完全绕开；
- 若窗口内关系变化很快，可把“按窗口一张图”改成“按时间步一张图”或“按若干子片段一张图”；
- 若想让图分支显式利用采样率，可把 ***rate_id*** 或相位编码并入节点动态上下文，而不仅仅交给门控层和检测器。

11 结论

回到用户最关心的问题，当前 `mra.py` 里的图插补模块可以被准确概括为：它先在训练集上拟合标准化器、采样率元数据和训练数据驱动的变量距离先验，再在每个窗口内根据均值、标准差、最近观测值和缺失比例生成一张自适应变量图，随后用这张图驱动三层 GCN 重构，并与频域专家通过门控融合，最终把补全结果送入检测器形成异常分数。

训练阶段更新的不只是图学习器和 GCN 参数，还包括门控层、频域专家和检测器；反过来，检测器损失也会回传影响图专家。测试阶段不再更新参数，但训练阶段得到的标准化统计量、图先验、采样率元数据、训练分数归一化参数和阈值都会继续生效。换言之，图学习模块既是前端插补器，也是后续异常检测分数形成过程中的结构性参与者，而不是一个与检测阶段割裂的独立预处理器。